Laboration 1 – Linjära datastrukturer och tidskomplexitet

Uppgift 1 Länkad lista

Min implementation av en länkad lista består av tre delar: ett interface (List), en nodklass (Node) och en klass (LinkedList).

Listan består av noder där varje nod innehåller ett element och en referens till nästa nod. Den första noden i listan refereras av variabeln head.

Klassen innehåller metoder för att lägga till, hämta, ändra och ta bort element. För att nå ett visst index måste listan traverseras från början, vilket påverkar prestandan.

AI-jämförelse

Jag testade att låta en generativ AI skapa en implementation av en länkad lista.

Den AI-genererade koden var mer avancerad och använde fler färdiga lösningar och strukturer. Den var ibland svårare att följa.

Min egen kod är mer direkt och steg-för-steg, där jag själv deklarerar variabler och bygger upp logiken. Det gör den lättare att förstå.

En likhet var att båda lösningarna använder noder och referenser för att koppla ihop listan, samt att båda implementerar metoder som add, get och remove.

Skillnaden är att AI-koden är mer generell, medan min kod är mer enkel och tydlig.

Det som var bra med AI-koden var att den visade andra sätt att strukturera koden. Det som var mindre bra var att den kunde kännas mer komplex än nödvändigt.

Jag valde att behålla min egen kod eftersom den är lättare att förstå.

I slutändan uppfyller både min kod och den AI-genererade koden målet med laborationen, vilket är att förstå hur en länkad lista fungerar och hur olika operationer påverkar tidskomplexiteten.

Uppgift 2 Insättning i listor Hypotes LinkedList

Jag tror att det går snabbast att lägga till ett element först i en länkad lista. Detta eftersom man bara behöver ändra referensen till första noden. Att lägga till i mitten eller sist kräver att man först går igenom listan, vilket tar längre tid.

kvot

Kvoten visar hur tiden förändras när listans storlek fördubblas. För insättning i mitten och sist ökar tiden ungefär proportionellt med listans storlek, vilket tyder på linjär tidskomplexitet.

Vissa kvoter varierar kraftigt, vilket kan bero på mätfel, bakgrundsprocesser och hur Java hanterar minne

Kvot (tabell)

	A	B	C	D	E	F
1	n	Kvot AL Först	Kvot LL Mitten	Kvot LL Sist	Kvot AL Mitten	Kvot AL Sist
2	40000/20000	1.94	2.06	2.14	0.80	0.34
3	80000/40000	0.67	1.90	4.70	1.60	1.00
4	160000/80000	2.14	5.29	2.41	2.34	96.07
5	320000/160000	1.86	2.57	3.28	1.73	0.01
6	640000/320000	2.17	0.92	2.60	2.08	0.56

Resultat (tabell)

	A	B	C	D	E	F	G
1	n	LL_Först	LL_Mitten	LL_Sist	AL_Först	AL_Mitten	AL_Sist
2	20000	86900	1027300	1941500	671100	342499	4100
3	40000	5300	2112100	4158700	1304899	272501	1400
4	80000	8900	4020100	19545200	878499	435101	1400
5	160000	17200	21278200	47152000	1882200	1016100	134500
6	320000	900	54753700	154474300	3493600	1754300	1599
7	640000	900	50334600	401647900	7588501	3654200	900

Analys

Resultaten för min egen LinkedList visar att insättning först i listan går klart snabbast. Tiderna för att lägga till i mitten är mycket högre, och att lägga till sist tar längst tid av alla tre fallen.

Detta stämmer väl med hypotesen. När ett element läggs till först behöver man bara ändra referensen till head, vilket är en mycket snabb operation. När ett element däremot ska läggas till i mitten eller sist måste programmet först gå igenom listan för att hitta rätt position.

Resultaten visar också att tidsåtgången ökar när listans storlek ökar, särskilt för insättning i mitten och sist. Insättning först ligger däremot kvar på mycket låga tider även när n blir större, vilket tyder på konstant tidskomplexitet (O(1)).

För insättning i mitten och sist ökar tiden mer i takt med listans storlek, vilket tyder på linjär tidskomplexitet (O(n)).

Under experimentet märkte jag att körningarna tog mycket lång tid, särskilt för de största liststorlekarna. Därför behövde jag justera experimentet för att få fram resultat inom rimlig tid. Detta kan påverka exakta värden något, men resultaten visar ändå tydliga skillnader mellan operationerna.

Resultaten stämmer därför överens med teorin om att insättning i början av en länkad lista är effektiv, medan insättning längre in i listan kräver mer arbete.

ArrayList

Resultaten för ArrayList visar att det går snabbast att lägga till ett element sist i listan. Detta stämmer med hypotesen eftersom element kan läggas till direkt utan att andra element påverkas.

När ett element läggs till först eller i mitten av listan måste alla efterföljande element flyttas för att göra plats. Detta gör att dessa operationer tar längre tid.

Resultaten visar också att tidsåtgången ökar när listans storlek ökar, särskilt för insättning först och i mitten. Detta tyder på att dessa operationer har linjär tidskomplexitet, ungefär O(n).

Insättning sist påverkas mycket mindre av listans storlek och är därför mer effektiv. Detta beror på att ArrayList är optimerad för att lägga till element i slutet av listan.

Resultaten stämmer därmed överens med teorin om att ArrayList är effektiv för insättning i slutet, men mindre effektiv för insättning i början eller mitten.

Uppgift 3 Iteratorer Hypotes

Jag tror att for-each-loopen kommer vara snabbare än en vanlig for-loop för en LinkedList. Detta eftersom for-each använder en iterator som går igenom listan direkt, medan en vanlig for-loop använder get(i), vilket kräver att listan traverseras varje gång ett element hämtas.

kvot

För den vanliga for-loopen ökar tiden kraftigt när listan växer, medan for-each-loopen håller en mer stabil tidsnivå. Detta visar tydligt skillnaden i effektivitet mellan metoderna.

Kvoterna visar att tiden för den vanliga for-loopen ökar mycket snabbare än för for-eachloopen, vilket tyder på att for-loopen har högre tidskomplexitet.

Kvot(tabell)

	A	B	C
1	n	Kvot_For_loop	Kvot_For_each
2	2000/1000	2.24	1.23
3	4000/2000	3.65	0.26
4	8000/4000	4.11	1.87
5	16000/8000	4.06	0.52
6	32000/16000	4.28	1.65

Resultat (tabell)

	A	B	C
1	n	For_loop	For_each
2	1000	4564000	722300
3	2000	10215100	886100
4	4000	37266000	230900
5	8000	153130599	430701
6	16000	621740200	222100
7	32000	2662209100	366600

Analys

Resultaten visar tydligt att for-each-loopen är mycket snabbare än den vanliga forloopen för LinkedList. Skillnaden ökar också kraftigt när listans storlek blir större.

Den vanliga for-loopen använder metoden get(i), vilket innebär att listan måste traverseras från början varje gång ett element hämtas. Detta gör att tidsåtgången ökar mycket snabbt när listan växer.

For-each-loopen använder istället en iterator som går igenom listan nod för nod, vilket är betydligt mer effektivt. Därför ligger tidsåtgången för for-each kvar på en låg nivå även när listan blir större.

Resultaten visar att den vanliga for-loopen får en mycket högre tidskostnad jämfört med for-each-loopen. Detta beror på att LinkedList inte är optimerad för indexbaserad åtkomst.

Resultaten stämmer därför väl överens med teorin om att iterator-baserad genomgång är mer effektiv för länkade listor än indexbaserad åtkomst.